

K.L.E SOCIETY'S

GUDLEPPA HALLIKERI BCA COLLEGE

HAVERI-581110

(Re-accredited by NAAC at "A" Grade)

(Affiliated To Karnataka University,Dharwad)



A Project Report on

"HOTEL MANAGEMENT SYSTEM"

Submitted by,

1.POOJA SANKANNAVAR-U02IG22S0119
2.RAKSHITA BARADUR-U02IG22S0021

Under the Guidance of

Prof. CHANDRAKANTH H MTech
K.L.E's G H BCA College
Haveri-581110

K.L.E SOCIETY'S

G H BCA COLLEGE

HAVERI-581110

(Re-accredited by NAAC at "A" Grade)

(Affiliated To Karnataka University, Dharwad)

Certificate

This is to certify that Miss/Mr. POOJA SANKANNAVAR with UUCMS U02IG22S0119 and Miss/Mr. RAKSHITA BARADUR with UUCMS U02IG22S0021had satisfactorily completed the project work entitled " HOTEL MANAGEMENT SYSTEM " for the partial fulfillment of Bachelor of Computer Application by the Karnataka University, Dharwad, during the academic year 2024-2025.

Prof. CHANDRAKANTH H
Signature of Guide

Prof. Venkatesh Kalal
Co-ordinator

Name of Examiners Signature with Date

1.

2.

ACKNOWLEDGEMENT

Salutation to our beloved and highly esteemed institute KLE's G.H BCA College, for having well qualified staff and labs furnished with necessary equipment.

I express my sincere thanks to our guide Prof.Chandrakanth H and project coordinators Prof.Shilpa Yadawad and Prof. Vinuta S for giving constant encouragement, support and valuable guidance throughout the course of the Project, without whose stable guidance this project report would not have been achieved.

I would like to thank our beloved Coordinator Prof. Venkatesh Kalal for giving me the opportunity and guidance to work for the Project.

I would like to extend my gratitude to all staff of K.L.E's G H BCA College for the help and support rendered to me. I have benefited a lot from the feedback, suggestions given by them.

I would like to extend my gratitude to all my family members and friends especially for their advice and moral support.

POOJA SANKANNAVAR

U02IG22S0119

RAKSHITA BARADUR

U02IG22S0021

ABSTRACT

The Hotel Management System is a web-based application designed to streamline and automate daily hotel operations through integrated modules for Room Booking, Table Ordering, Staff Management, and Stock Management. It offers dedicated interfaces for both admin and users, enabling efficient management and personalized guest experiences. The system allows guests to book rooms with options and single or double beds, while also specifying special requirements. It facilitates smooth table ordering with menu viewing, food ordering, billing, and pending order tracking. Staff records and salaries are managed through the Staff Management module, and the Stock Management module ensures real-time inventory tracking, purchase handling, and expense recording. By minimizing manual work and providing real-time updates and reporting, the system significantly enhances operational efficiency and service quality.

INDEX

SLNO	TITLE	PAGENO
CHAPTER1	**INTRODUCTION**	
1.1	INTRODUCTION	1
1.2	EXISTING SYSTEM	2
1.3	PROPOSED SYSTEM	2
1.4	PROBLEM STATEMENT	3
1.5	OBJECTIVES	3
1.6	ADVANTAGES	3
CHAPTER2	**HARDWARE AND SOFTWARE REQUIREMENTS**	
2.1	HARDWARE REQUIREMENTS	4
2.2	SOFTWARE REQUIREMENTS	4
CHAPTER3	**SYSTEM DESIGN**	
3.1	DESIGN OVERVIEW	5
3.2	SYSTEM ARCHITECHTURE	5-6
3.3	MODULE DESCRIPTION	7-8
3.4	METHODOLOGY	9-10
3.5	TABLE CREATION	11-16
3.6	E-R DIAGRAM	17
3.7	USECASE DIAGRAM	18
CHAPTER4	**IMPLEMENTATION**	
4.1	PSEUDO CODE	19-32
CHAPTER5	**SYSTEM TESTING**	
5.1	TESTING OVERVIEW	33
5.2	SYSTEM TESTING	33
5.3	TESTCASES	34
CHAPTER6	**RESULTS AND SNAPSHOTS**	35-40
	CONCLUSION	
	FUTURE ENHANCEMENT	
	REFERENCES	

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

This Hotel Management System is a comprehensive web application featuring both admin and user interfaces. designed to streamline and automate the day-to-day operations of a hotel. The project integrates such as Room Booking, Table Ordering, Staff Management, and Stock Management to enhance efficiency and user experience. The Room Booking System allows customers to view available rooms and make reservations seamlessly. It includes features to add rooms, categorize them as A/C or non-A/C, and specify bed types like single or double. Guests can also mention special requirements while booking, making the experience more personalized. The Table Ordering System enables users to view the food menu, place table orders, and enjoy a smooth table ordering experience. This module supports adding tables, adding food items, settle bill and printing bills and pending orders. It automates bill generation by calculating taxes and total amounts based on selected menu items. The Staff Management module keeps records of all hotel employees and salary information. Stock Management monitors inventory levels, handles purchase entries, and records item related expenses. This ensures resource availability while helping to control operational costs. Overall, the system is designed to deliver real-time updates, minimize manual work, and offer insightful reporting across all hotel functions.

1.2 EXISTING SYSTEM

The existing hotel management system is mostly manual and paper-based. Room bookings are written down in registers, which can lead to double bookings or missing records. Food orders are taken by waiters and manually passed to the kitchen, causing delays and confusion. Staff records and salary details are stored in files, making updates difficult. Stock management is done without proper tracking, leading to shortages or wastage.

Overall, the system is slow, less accurate, and hard to manage.

DISADVANTAGE

- Inefficient booking management
- Poor guest experience
- Poor staff management
- Inefficient food ordering management
- Inaccurate table billing and reporting
- Poor stocks management

1.3 PROPOSED SYSTEM

The proposed Hotel Management System is a web application designed to automate and improve the efficiency of hotel operations. It allows users to view and book rooms with options like A/C or non-A/C and different bed categories. The system simplifies table ordering by showing the food menu and managing orders digitally. Billing is automated with accurate tax calculations and easy bill printing. Staff records and salary management for better control. Stock management tracks purchases and expenses, providing detailed reports for effective decision-making.

1.4 PROBLEM STATEMENTS

Managing hotel operations manually is time-consuming and prone to errors, leading to inefficiencies. Booking rooms, handling food orders, staff records, and stock management often lack coordination and automation. This causes delays in service, inaccurate billing, and poor

1.5 OBJECTIVES

- To Automate room booking
- To Automate table ordering
- To Automate staff management
- To Automate Stock management

1.6 ADVANTAGES

- This project aims to reduce manual work
- Improve a room management
- Minimize errors and provide a better experience for both guests and hotel staff
- Enhance table ordering
- Reduced staff salary maintenance
- Enhance stock management

CHAPTER 2

HARDWARE AND SOFTWARE REQUIREMENTS

2.1 HARDWARE REQUIREMENTS

- Processor Core i3
- RAM 8GB
- Hard Disk 512GB

2.2 SOFTWARE REQUIREMENTS

- Operating System Windows 10/11
- Frontend HTML, BOOTSTRAP, CSS
- Backend PYTHON
- Database SQLite (db.sqlite3)
- Framework DJANGO
- Web server XAMPP
- Idle PYCHARM

CHAPTER 3

SYSTEM DESIGN

3.1 DESIGN OVERVIEW

The Hotel Management System is a comprehensive web application with distinct admin and user interfaces, aimed at automating key hotel operations. It includes four major modules: Room Booking, Table Ordering, Staff Management, and Stock Management. Guests can view available rooms and book based on preferences such as a/c or Non-a/c, single or double beds, and additional requirements. The Table Ordering module lets users explore the food menu, place table-specific orders, and manage bills, including tax calculations and printing. Admins can add food items, manage tables, and monitor pending items. The Staff Management feature maintains employee records and handles salary tracking efficiently. Stock Management oversees item purchases and expenses, ensuring real-time inventory monitoring. The system also generates detailed reports to support datadriven decision-making.

3.2 SYSTEM ARCHITECTURE

The Hotel Management System follows a structured client-server architecture, where both admin and user interfaces interact through a centralized Django backend. Admin functionalities include managing rooms, items, staff, tables, purchases, and expenses, all linked to a central database. Users can register, log in, view menus, order tables, and book rooms, with all actions recorded in the same database for consistency. This architecture ensures seamless data flow, real-time updates, and efficient hotel operations through modular integration.

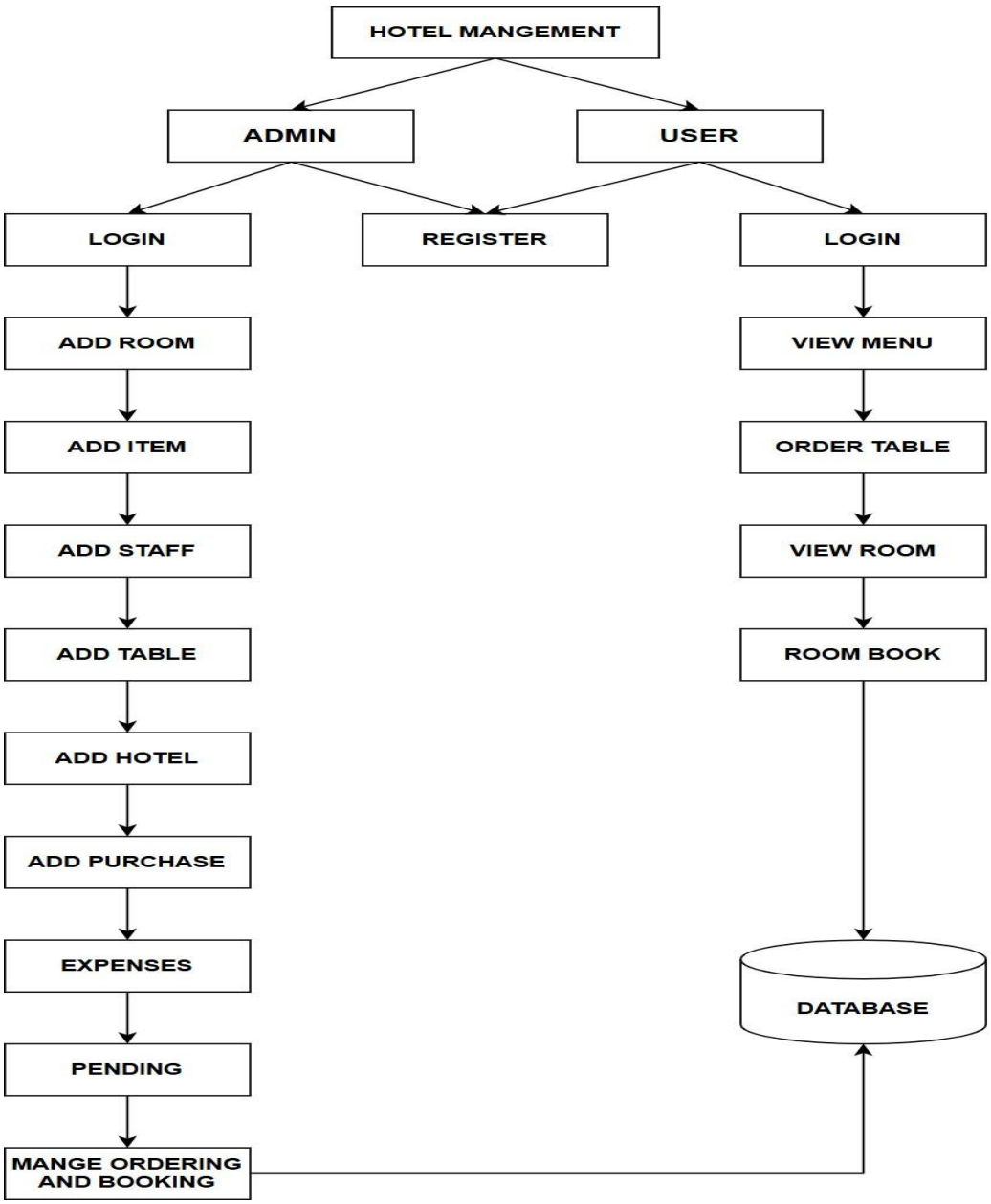


Fig 3.2.1: System Architecture

3.3 MODULE DESCRIPTION

1. **ADMIN MODULE**
2. **USER MODULE**

1 ADMIN MODULE

• **Register:**

Allows admin users to create a secure account for system access**.**

• **Login:**

Authenticates admin credentials to access the admin dashboard securely.

• **Add Table:**

Enables the admin to create and manage tables, category, table name.

• **Add Room:**

Allows the admin to add new hotel rooms with type, pricing, and availability settings.

• **Add Staff:**

Provides functionality of staff name and salary.

• **Add Purchase:**

Lets the admin record and track purchases made for hotel operations

• **Add Items:**

Enables the admin to manage inventory items including stock categories.

• **Add Hotel**

Define hotel details like name, location, and contact info.

• **Expenses**

The daily hotel operational and maintenance expenses.

• **Pending**

View pending bookings, orders, and verification tasks.

• **User Orders/Bookings**

Monitor room and table bookings made by users.

2 USER MODULE

• **Register:**

Allows users to create a personal account to access hotel services.

• **Login:**

Authenticates users to securely access their dashboard and features.

• **View Menu:**

Displays a list of available food and drink for selection.

• **Table Ordering:**

Enables users to place food orders from their reserved table.

• **View Room:**

Shows available rooms with details like type, price, and amenities.

• **Room Booking:**

Lets users book selected rooms by choosing dates and preferences.

3.4 METHODOLOGY

1. HTML (Hyper Text Markup Language)

HTML is used to structure the core content of the Hotel Management System, including booking forms, room listings, and user dashboards. It provides the framework for displaying text, images, links, and interactive elements on the web pages. HTML ensures that the content is well-organized and can be easily interpreted by both users and search engines.

2. CSS (Cascading Style Sheets)

CSS is employed to style the HTML elements, giving the website its visual design and layout. It defines the look and feel of the site by controlling colors, fonts, spacing, and positioning of elements. CSS also enhances the responsiveness of the site, ensuring it functions well across devices such as desktops, tablets, and smartphones.

3. Bootstrap (CSS framework)

Bootstrap is a front-end framework that simplifies the development of responsive and mobile-first web pages. It provides a wide range of pre-designed components, such as navigation bars, forms, and buttons, which can be easily customized. By using Bootstrap, the Hotel Management System is made compatible with various screen sizes, improving the user experience.

4. Python (Django)

Python, with the Django framework, is used to handle the server-side logic of the application, ensuring smooth data processing and interactions. Django's built-in features, like routing, authentication, and ORM (Object-Relational Mapping), help manage the database and simplify backend development. It ensures the application is secure, scalable, and easy to maintain.

5. SQLite

SQLite is a lightweight, file-based relational database used by default in Django projects. It stores data in a single file named (db.sqlite3). It is easy to set up and does not require a separate server. SQLite supports standard SQL queries and works well for small to medium web applications. It is ideal for development and testing purposes in Django.

6.XAMPP (Server)

XAMPP is a local development environment that bundles Apache, MySQL, and PHP to simulate a web server on the local machine. It allows developers to test the PHP and MySQL-based Hotel Management System before deployment. XAMPP ensures that the server runs smoothly during the development phase, providing a testing platform for the system.

7.PyCharm (Integrated Development Environment (IDE))

PyCharm is a powerful Integrated Development Environment (IDE) used for Python and Django development. It provides features like code completion, debugging tools, and version control integration, making it easier to write and maintain the backend code. PyCharm enhances the development experience by offering advanced tools to streamline the creation of the Hotel Management System.

3.5 TABLE CREATION

ADMIN

ADMIN REGISTER

Structure | Data | Constraints | Indexes | Triggers | DDL

new | Table name: food_app_admin_register | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer	🔑				🔒			NULL
2	admin_name	varchar (20)					🔒			NULL
3	mobile_number	varchar (10)					🔒			NULL
4	admin_id	varchar (10)					🔒			NULL
5	password	varchar (20)					🔒			NULL

Fig 3.5.1: Admin Register

ADD HOTEL

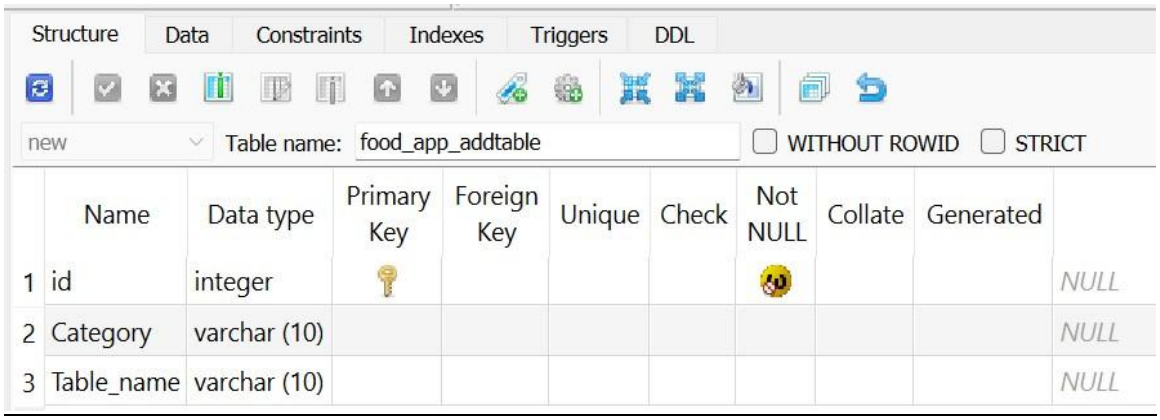
Structure | Data | Constraints | Indexes | Triggers | DDL

new | Table name: food_app_addcompany | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer	🔑				🔒			NULL
2	Company_name	varchar (20)								NULL
3	Company_address	varchar (50)								NULL
4	Company_GST	varchar (10)								NULL
5	Company_number	varchar (10)								NULL

Fig 3.5.2: Add Hotel

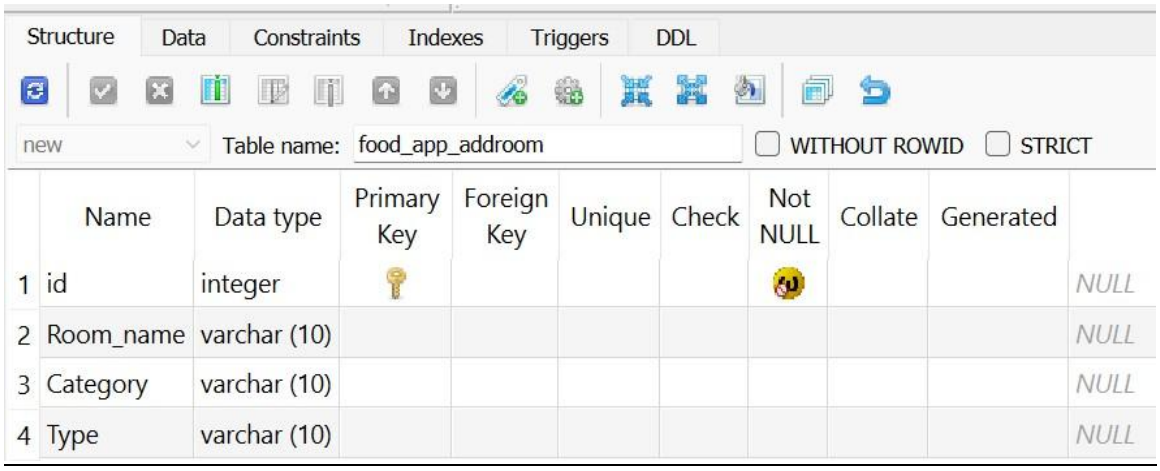
ADD TABLE



	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer								NULL
2	Category	varchar (10)								NULL
3	Table_name	varchar (10)								NULL

Fig 3.5.3: Add Table

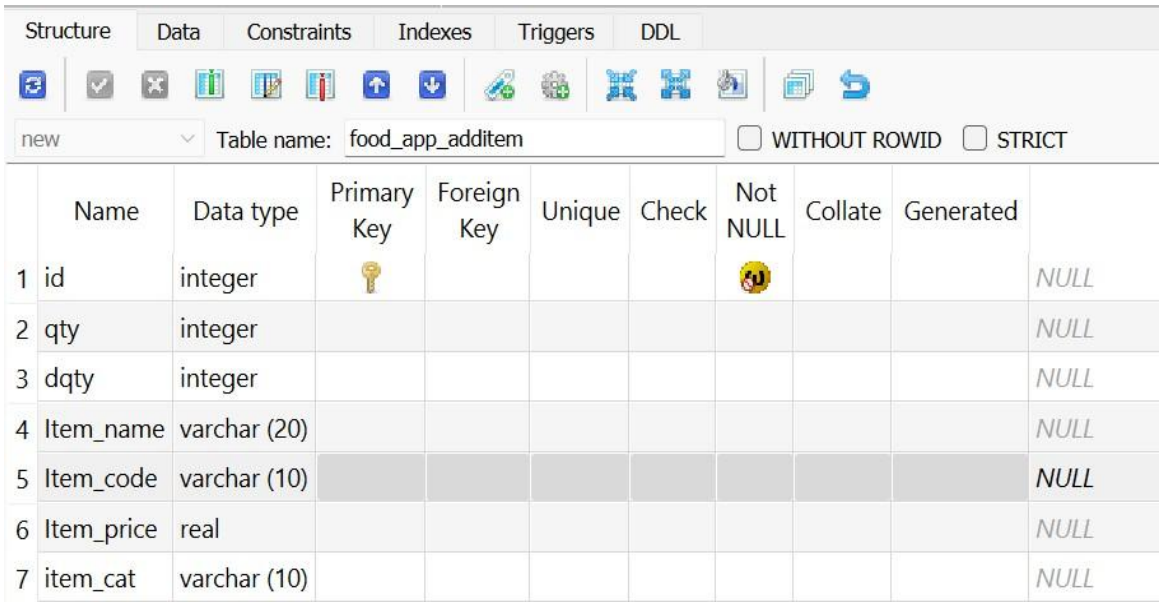
ADD ROOM



	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer								NULL
2	Room_name	varchar (10)								NULL
3	Category	varchar (10)								NULL
4	Type	varchar (10)								NULL

Fig 3.5.4: Add Room

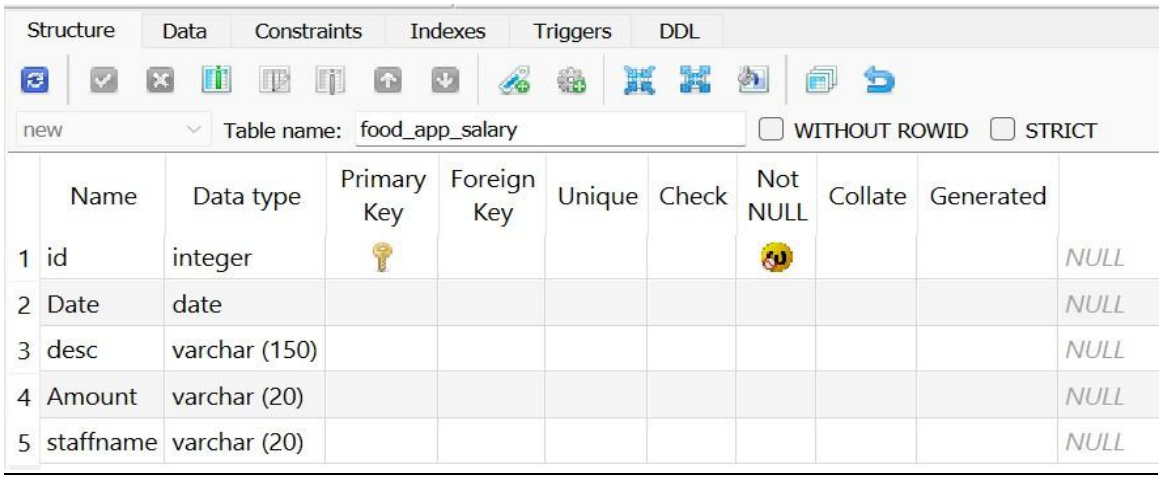
ADD ITEMS



	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer								NULL
2	qty	integer								NULL
3	dqty	integer								NULL
4	Item_name	varchar (20)								NULL
5	Item_code	varchar (10)								NULL
6	Item_price	real								NULL
7	item_cat	varchar (10)								NULL

Fig 3.5.5: Add Items

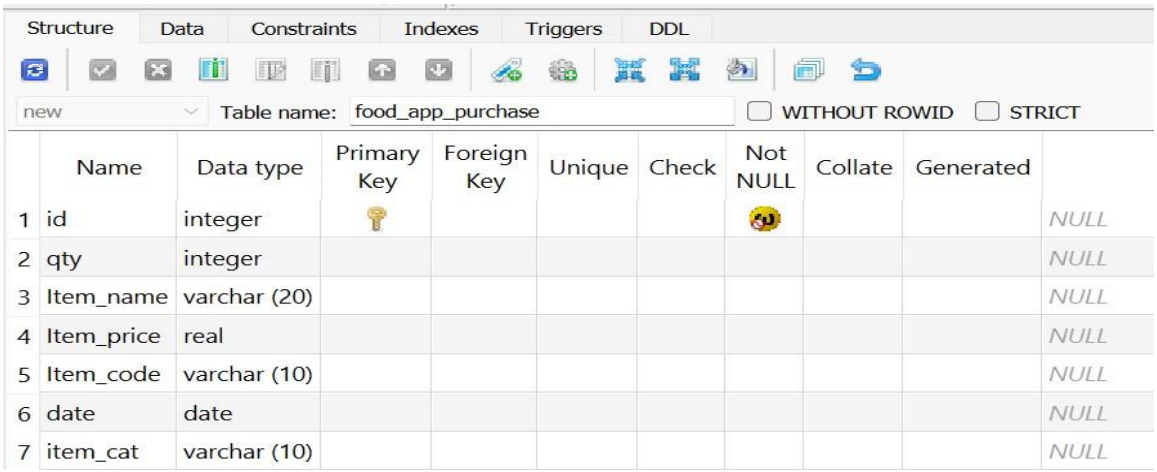
ADD STAFF



	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer								NULL
2	Date	date								NULL
3	desc	varchar (150)								NULL
4	Amount	varchar (20)								NULL
5	staffname	varchar (20)								NULL

Fig 3.5.6: Add Staff

ADD PURCHASE



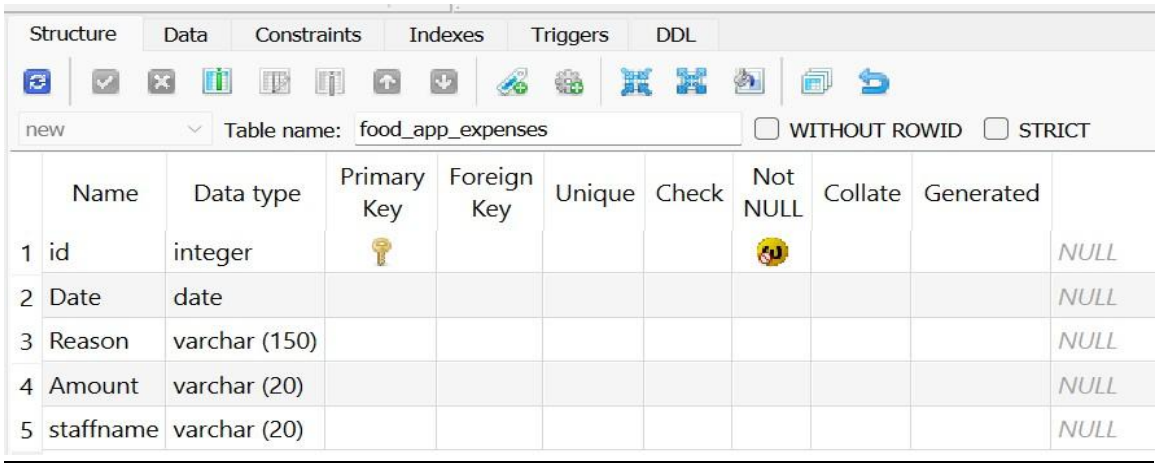
Structure | Data | Constraints | Indexes | Triggers | DDL

new | Table name: food_app_purchase | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer								NULL
2	qty	integer								NULL
3	Item_name	varchar (20)								NULL
4	Item_price	real								NULL
5	Item_code	varchar (10)								NULL
6	date	date								NULL
7	item_cat	varchar (10)								NULL

Fig 3.5.7: Add Purchase

EXPENSES



Structure | Data | Constraints | Indexes | Triggers | DDL

new | Table name: food_app_expenses | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer								NULL
2	Date	date								NULL
3	Reason	varchar (150)								NULL
4	Amount	varchar (20)								NULL
5	staffname	varchar (20)								NULL

Fig 3.5.8: Expenses

PENDING

Structure | Data | Constraints | Indexes | Triggers | DDL

new | Table name: food_app_pendingbill | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer								NULL
2	date	date								NULL
3	billNo	integer								NULL
4	phone	varchar (10)								NULL
5	name	varchar (10)								NULL
6	status	varchar (20)								NULL
7	amount	real								NULL
8	table	varchar (10)								NULL

Fig 3.5.9: Pending

USER

USER REGISTER

Structure | Data | Constraints | Indexes | Triggers | DDL

new | Table name: food_app_user_register | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer								NULL
2	user_name	varchar (20)								NULL
3	mobil_number	varchar (10)								NULL
4	password	varchar (20)								NULL

Fig 3.5.10: User Register

TABLE ORDER

Structure | Data | Constraints | Indexes | Triggers | DDL

new | Table name: food_app_orders | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer	🔑				🔒			NULL
2	bill_no	varchar (10)								NULL
3	bill_date	date					🔒			NULL
4	bill_time	time								NULL
5	item_code	varchar (10)								NULL
6	item_name	varchar (10)								NULL
7	price	decimal								NULL
8	gst	decimal								NULL
9	qty	integer					🔒			NULL
10	tax_amt	decimal								NULL
11	total_gst	decimal								NULL
12	total	real								NULL
13	item_cat	varchar (10)								NULL
14	table_name	varchar (10)								NULL

Fig 3.5.11: Table Order

ROOM BOOK

Structure | Data | Constraints | Indexes | Triggers | DDL

new | Table name: food_app_roombooking | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	integer	🔑				🔒			NULL
2	first_name	varchar (10)					🔒			NULL
3	email	varchar (254)					🔒			NULL
4	mobile_number	varchar (15)					🔒			NULL
5	check_in	date					🔒			NULL
6	check_out	date					🔒			NULL
7	room_type	varchar (20)					🔒			NULL
8	category	varchar (20)					🔒			NULL
9	last_name	varchar (10)					🔒			NULL

Fig 3.5.12: Room Book

3.6 E-RDIAGRAM

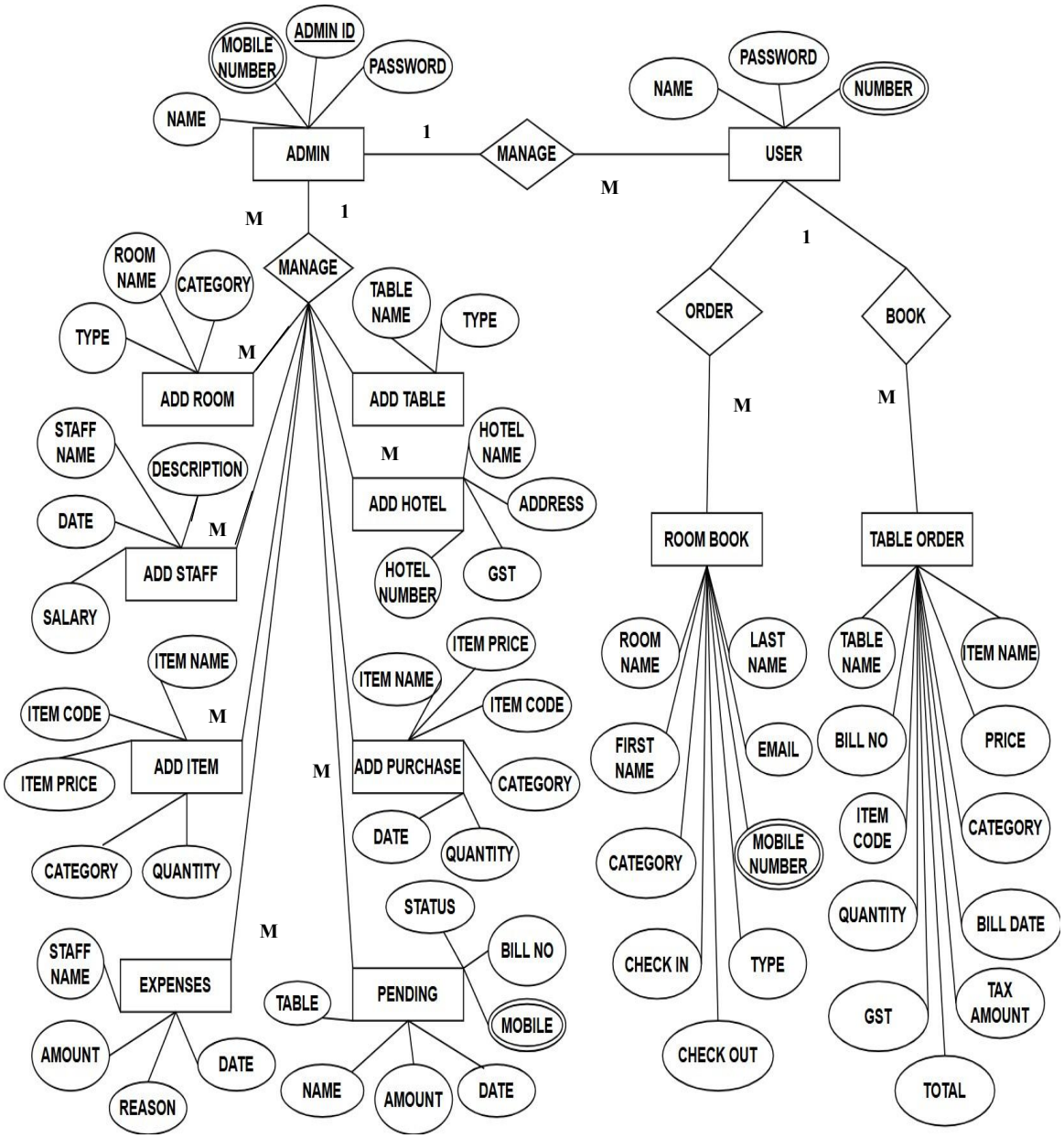


Fig 3.6.1: E R Diagram

3.7 USECASE DIAGRAMS

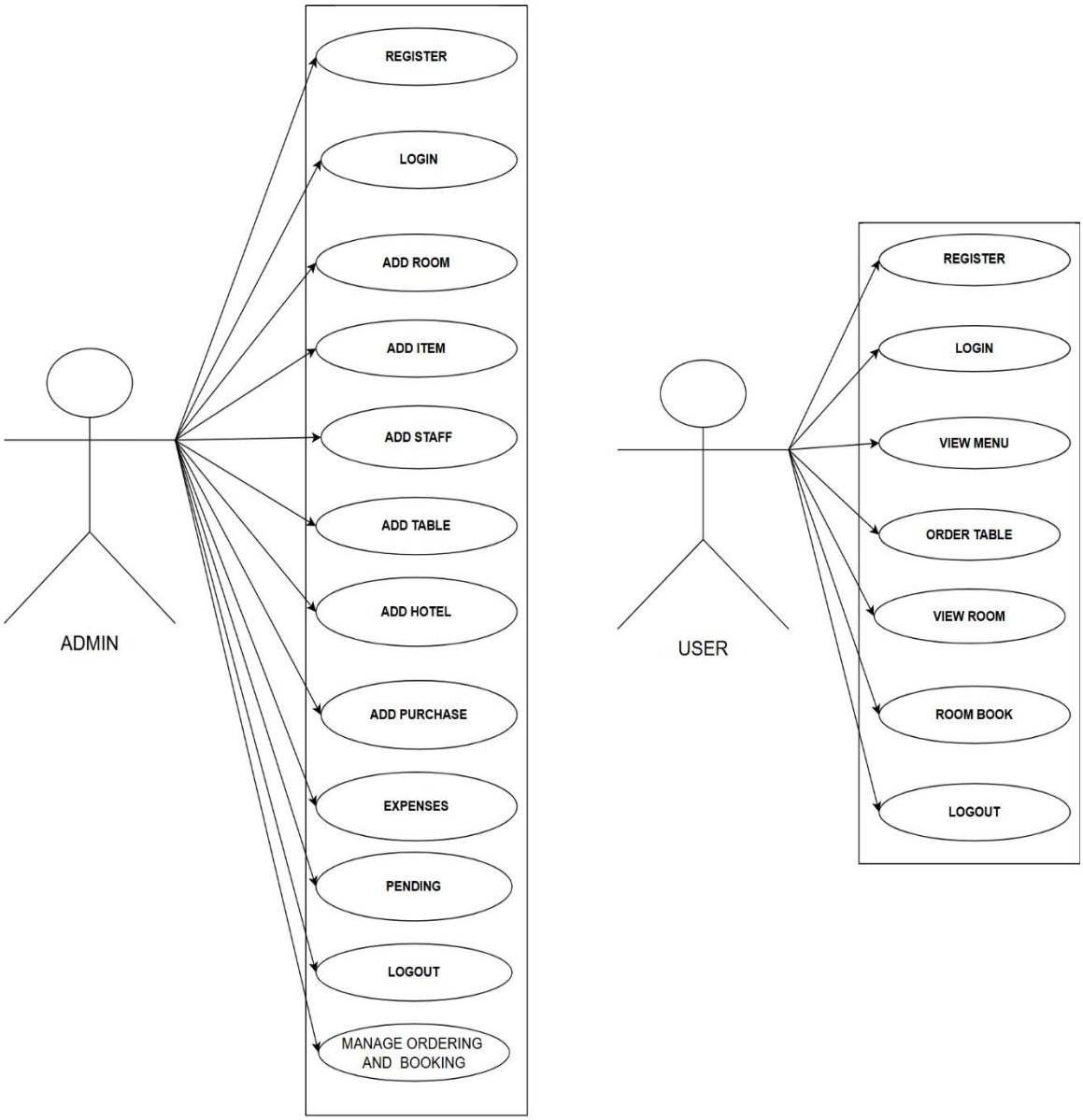


Fig 3.7.1: Use Case Diagram

CHAPTER 4

IMPLEMENTATION

4.1 PSEUDO CODE

ADMIN

ADMIN REGISTER

Function admin_reg(request):

IF the HTTP request method is POST THEN:

Retrieve 'adminname' from the POST data → store in admin_name

Retrieve 'number' from the POST data → store in mobile_number

Retrieve 'adminid' from the POST data → store in admin_id

Retrieve 'password' from the POST data → store in password

Create a new admin_register object with:

- admin_name = admin_name
- mobile_number = mobile_number
- admin_id = admin_id
- password = password

Save the admin_register object to the database

Render and return the 'adminlogin.html' template

ENDIF

END Function

ADMIN LOGIN

Import admin_register model from models

Function admin_login(request):

If the request method is POST:

Get the value of 'adminname' from the request and assign it to admin_name

Get the value of 'password' from the request and assign it to password

Search the admin_register table for a record where:

admin_name matches admin_name

AND password matches password

Store the first matching result in the variable admin

If admin is not None:

Save admin.admin_name in the session with the key 'adminname'

Return the 'admin_home.html' template

Else:

Return a plain text response: 'invalid username or password'

End Function

ADD HOTEL

Function add_company(request):

IF the HTTP request method is POST THEN:

Retrieve 't1' from the POST data → store in company_name

Retrieve 't2' from the POST data → store in company_address

Retrieve 't3' from the POST data → store in company_gst

Retrieve 't4' from the POST data → store in company_number

Create a new AddCompany object with:

- Company_name = company_name
- Company_address = company_address
- Company_GST = company_gst
- Company_number = company_number

Save the object to the database (handled automatically by `.create()`)

Render and return 'add_company.html' template with a message:

- 'msg' = 'Added'

ELSE:

Render and return 'add_company.html' template without message

END Function

ADD TABLE

Function add_table(request):

IF the HTTP request method is POST THEN:

Retrieve 't1' from the POST data → store in table_name

Retrieve 't2' from the POST data → store in cat (category)

Create a new AddTable object with:

- Table_name = table_name

- Category = cat

Save the object to the database (handled by .create()

Render and return 'add_table.html' template with:

- message 'msg' = 'Added'

ELSE:

Render and return 'add_table.html' template without any message

END Function

ADD ROOM

Function add_room(request):

IF the HTTP request method is POST THEN:

Retrieve 't1' from POST data → store in Room_name

Retrieve 't2' from POST data → store in Type

Retrieve 't3' from POST data → store in Category

Create a new AddRoom object with:

- Room_name = Room_name

- Type = Type

- Category = Category

Save the object to the database (automatically handled by .create())

Render and return 'add_room.html' template with:

- message 'msg' = 'Added'

ELSE:

Render and return 'add_room.html' template without any message

END Function

ADD ITEMS

Function add_item(request):

IF the HTTP request method is POST THEN:

Get the current date and time → store in 'now'

Format the current date to "YYYY-MM-DD" format → store in 'con_date'

Retrieve form data from POST request:

- 't2' → item_name

- 't1' → item_code

- 'qty' → qty

- 't4' → item_price

- 't5' → category

IF qty is an empty string THEN:

Set qty = None (meaning no quantity provided)

ELSE:

Convert qty to integer

Create a new AddItem object with the following fields:

- Item_name = item_name

- qty = qty

- Item_price = item_price

- Item_code = item_code

- item_cat = category

Save the object to the database (handled by `.create()`)

Render and return the template 'add_item.html' with message:

- msg = 'Item added successfully!' ELSE:

Render and return the template 'add_item.html' without any message

END Function

ADD STAFF

Function saveSalary(request):

IF the HTTP request method is POST THEN:

Retrieve 'date' from POST data → store in date_str

TRY to convert date_str to a date object using format "YYYY-MM-DD":

IF successful → store in 'date'

EXCEPT ValueError (invalid format):

Print "Invalid date format."

Redirect to 'adminExpenses' page

Retrieve the following from POST data:

- 'amt' → amount

- 'reason' → reason

- 'salaryDetails' → salaryDetails (staff name)

Create a new Salary object with:

- Date = date

- desc = reason

- Amount = amount

- staffname = salaryDetails

Save the object to the database (handled by `.create()`)

ELSE:

Print "No username found in session."

Redirect to 'salary' page

Render and return the 'salary.html' template

END Function

ADD PURCHASE

Function purchase(request):

IF the HTTP request method is POST THEN:

Get current date and time → store in 'now'

Format 'now' to "YYYY-MM-DD" format → store in 'con_date'

Retrieve the following POST data:

- 't2' → item_name
- 't1' → item_code
- 'qty' → qty (convert to integer)
- 't4' → item_price
- 'selling' → selling
- 't5' → item_cat TRY:

Search for an existing item in AddItem where Item_code equals item_code

IF item is found:

Add the new qty to the existing qty

Save the updated item

Set message = "Item updated successfully with new quantity!"

EXCEPT AddItem.DoesNotExist:

Create a new AddItem object with:

- Item_code = item_code
- Item_name = item_name
- qty = qty
- Item_price = selling
- item_cat = item_cat

Set message = "New item added successfully!"

ENDIF

END Function

EXPENSES

Function saveExpenses(request):

IF the HTTP request method is POST THEN:

Retrieve 'date' from POST data → store in date_str

TRY:

Convert date_str to a date object in format "YYYY-MM-DD" → store in date

EXCEPT ValueError:

Print "Invalid date format."

Redirect to 'adminExpenses' page

Retrieve the following from POST data:

- 'amt' → amount
- 'reason' → reason

Create a new Expenses object with:

- Date = date
- Reason = reason
- Amount = amount Save the object to the database ELSE:

Print "No username found in session."

Redirect to 'addExpenses' page

Render and return the 'expenses.html' template

END Function

PENDING

Function pending_bills_view(request):

Fetch all table records from AddTable model → store in 'table'

IF the HTTP request method is POST THEN:

Retrieve the following form data from POST request:

- 'date' → con_date
- 'billNo' → bill_no

- 'name' → name

- 'phone' → phone (or None if empty)

- 'table' → table

- 'status' → status

- 'amount' → amount

Create a new pendingBill record with:

- billNo = bill_no

- name = name

- phone = phone

- table = table

- status = status

- date = con_date

- amount = amount

Redirect to the 'pending_bills' page

ELSE:

(Missing from the code but implied):

- Retrieve all pending bills and calculate the grand total

- Store results in 'bills' and 'grand_total'

Render and return the 'pending_bills.html' template with:

- bills = bills

- table = table

- grand_total = grand_total

END Function

USER

USER REGISTER

Function user_reg(request):

IF the HTTP request method is POST THEN:

Retrieve the following data from POST request:

- 'username' → user_name

- 'number' → mobil_number

- 'password' → password

Create a new user_register object with:

- user_name = user_name

- mobil_number = mobil_number

- password = password

Save the new user_register object to the database

Render and return 'userlogin.html' template

END Function

USER LOGIN

Function user_login(request):

IF the HTTP request method is POST THEN:

Retrieve the following data from the POST request:

- 'username' → user_name

- 'password' → password

Query the user_register table:

- Filter for a user where user_name equals user_name AND password equals password

- Get the first matching record → store in user

IF a matching user is found THEN:

Set session variable 'username' = user's user_name

Render and return the 'usernav.html' template

ELSE:

Return an HTTP response with the message:

"invalid username or password"

END Function

TABLE ORDER

Function save_order(request):

Get the current datetime without microseconds → now

Format the current time as "HH:MM:SS" → current_time

IF current hour is less than 2 THEN: Set bill_date as the previous day's date ELSE:

Set bill_date as the current date

Parse JSON data from request body → store in 'data'

Extract:

- billDetails from data
- billItems from data From billDetails:
- Get table_name
- Get bill_no

FOR each item in billItems DO:

Extract from item:

- item_code
- item_cat
- item_name
- price (convert to float)
- qty (convert to int)
- tax_amt (convert to float)
- total (convert to float)

Create a new record in ItemOrder with:

- table_name, bill_no, bill_date, bill_time
- item_code, item_cat, item_name
- price, qty, tax_amt, total

Also create a new record in Orders with the same data

END Function

ROOM BOOK

Function register_room(request):

IF the HTTP request method is POST THEN:

Create a new RoomBooking object with the following fields:

- first_name = request.POST['first_name']
- last_name = request.POST['last_name']
- email = request.POST['email']
- mobile_number = request.POST['mobile_number']
- check_in = request.POST['check_in']
- check_out = request.POST['check_out']
- room_type = request.POST['room_type']
- category = request.POST['category']

Save the object to the database (handled by .create())

Redirect to the 'booking_success' page

ELSE:

Render and return the 'room_register.html' template

END Function

CHAPTER 5

SYSTEM TESING

5.1 TESTING OVERVIEW

In this Hotel Management System project, both black box and white box testing approaches are applied. Black box testing is used to test the system's functionality from the user's perspective—such as room booking, table ordering, and billing—without checking internal code. White box testing is adopted by the developers to test internal logic, code flow, and conditions like validations and calculations (e.g., tax computation in billing). Unit tests are used to check individual components, while integration testing ensures modules like booking and billing work together. These methods help ensure the system is robust, accurate, and userfriendly.

5.2 SYSTEM TESTING

System testing is the process of testing the complete and integrated Hotel Management System to verify that it meets the specified requirements. It ensures all modules—Room Booking, Table Ordering, Staff, and Stock Management—work together as intended. This phase is important because it validates the end-to-end functionality from a user’s perspective. It helps identify any integration issues, performance gaps, or unexpected behaviour before deployment. Effective system testing ensures the software is reliable, user-friendly, and ready for real-world use.

5.3 TEST CASES

FUNCTION	INPUTS	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT
Register (Admin and User)	Correct username, number and password	Successful Register	Successful Register	Success
Register (Admin and User)	Incorrect username but correct number and password	Invalid username	Invalid username	Fail
Login (Admin and User)	Correct username and password	Successful login	Successful login	Success
Login (Admin and User)	Incorrect username and password	Invalid username and password	Invalid username and password	Fail
Room Booking (User)	valid room book detail and user information	Booking confirm	Booking confirm	Success
Room Booking (User)	Room not available	Booking not confirm	Booking not confirm	Fail
Table Order (User)	valid room book detail and user information	Order confirm	Order confirm	Success
Table Order (User)	Table not available	Table not confirm	Table not confirm	Fail

Fig 5.3.1: Test Case

CHAPTER 6

RESULTS AND SNAPSHOTS

HOMEPAGE

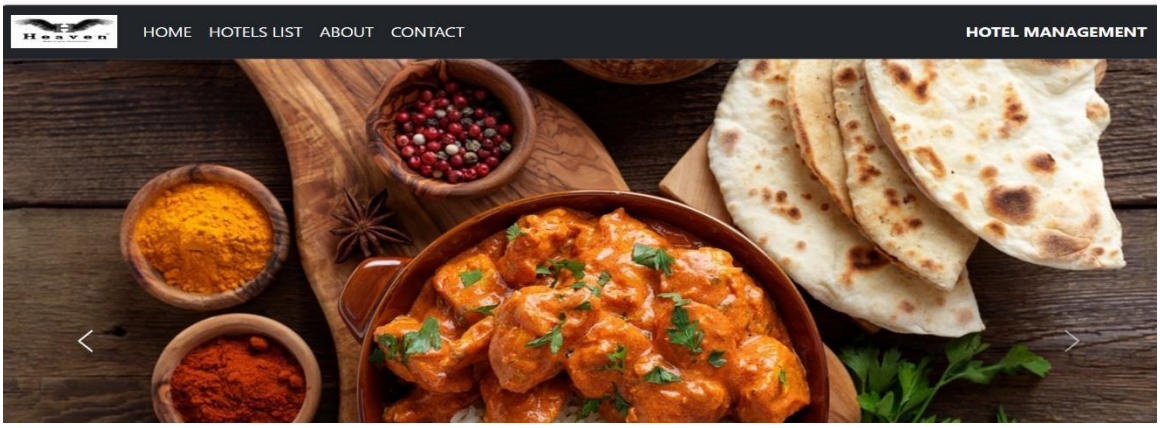


Fig 6.1: Homepage

HOTELS LIST

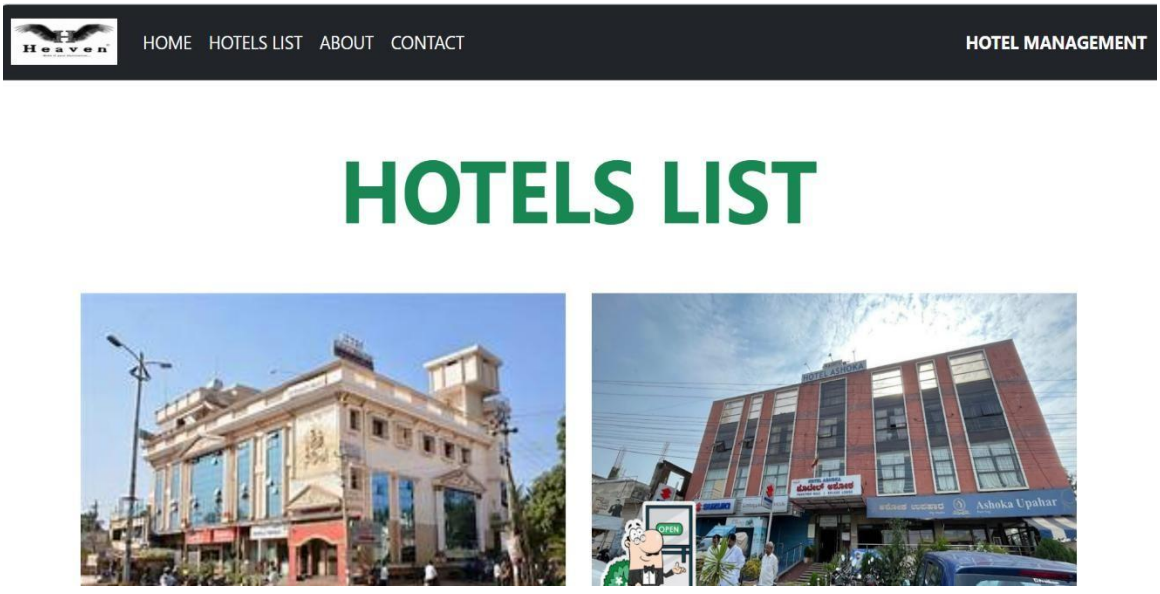


Fig 6.2: Hotels List

VIEW MENU

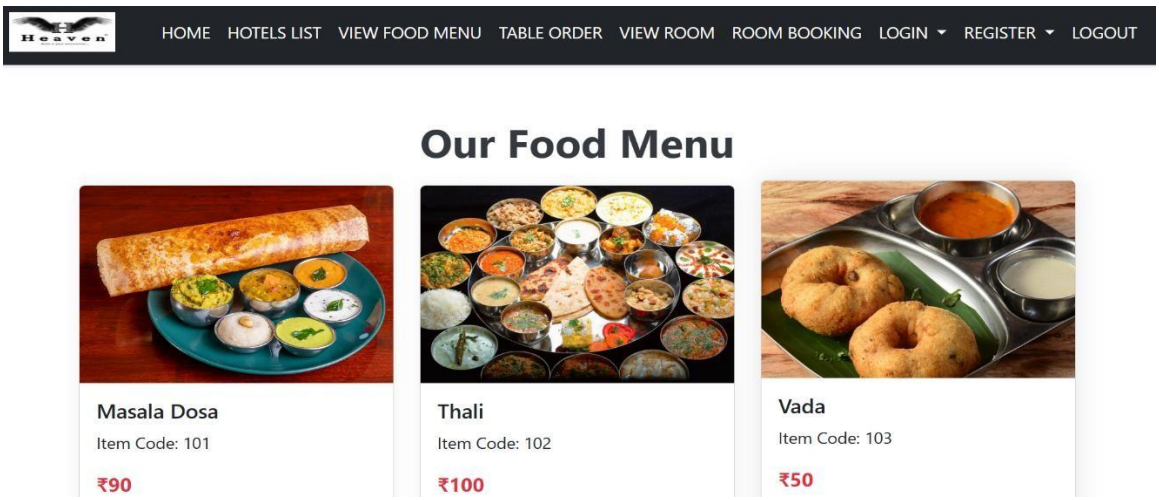


Fig 6.3: View Menu

VIEW ROOM

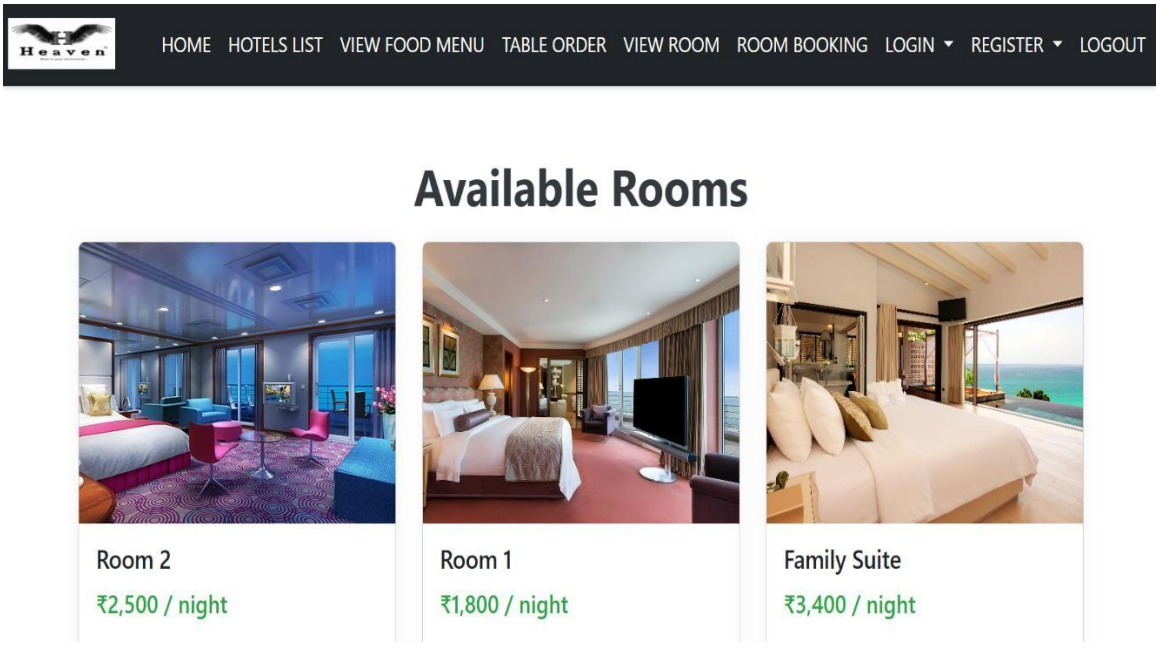


Fig 6.4: View Room

WELCOME PAGE

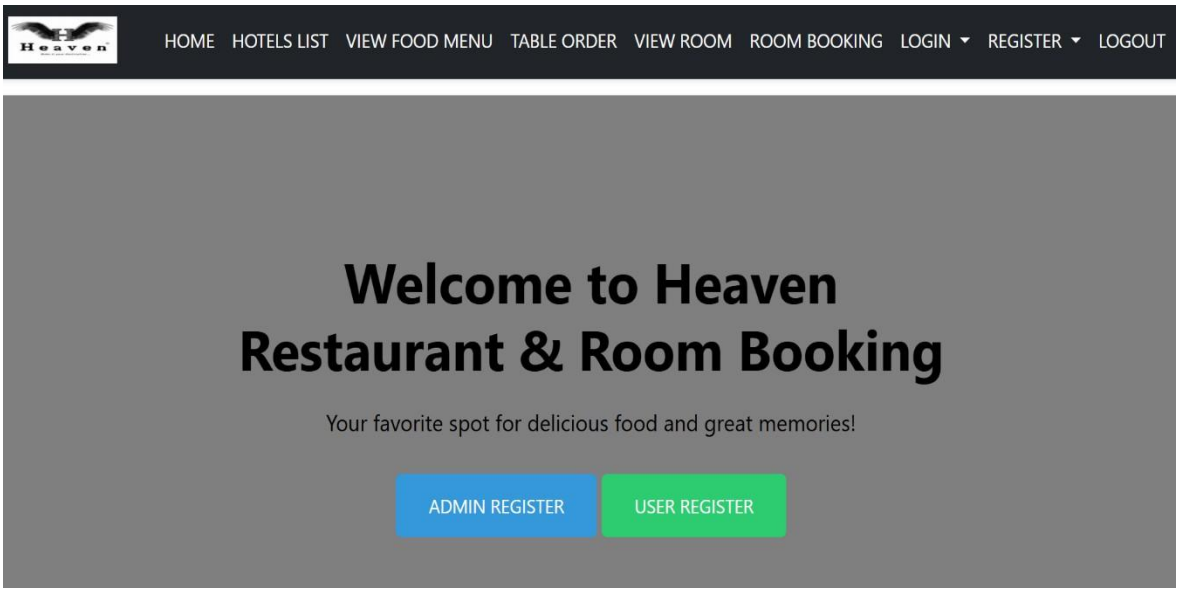


Fig 6.5: Welcome Page

ADMIN REGISTER

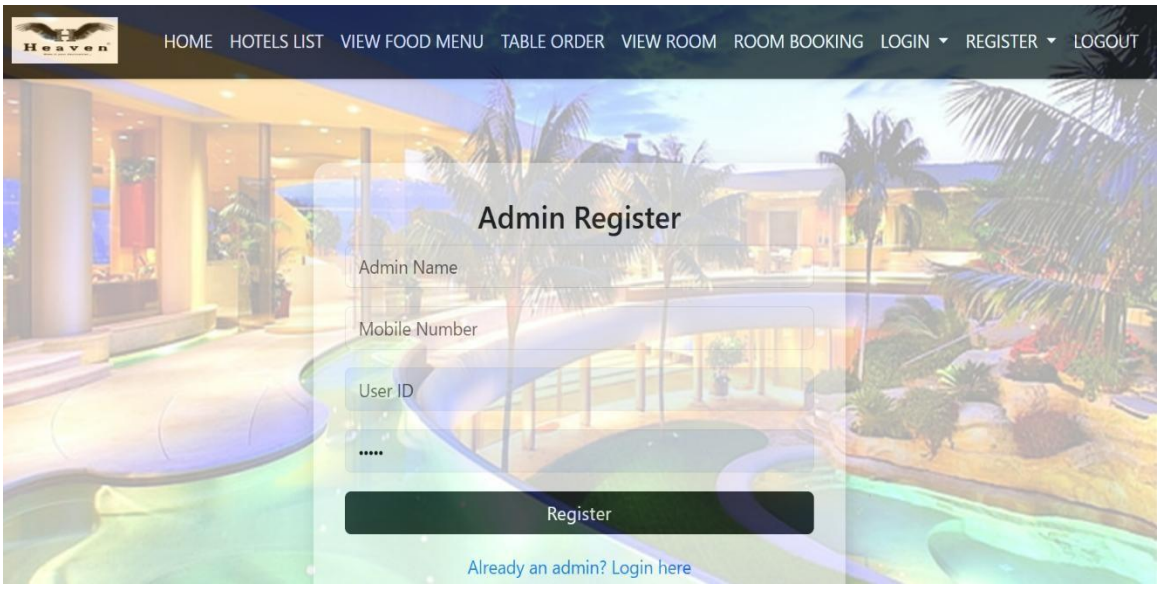


Fig 6.6: Admin Register

ADMIN LOGIN

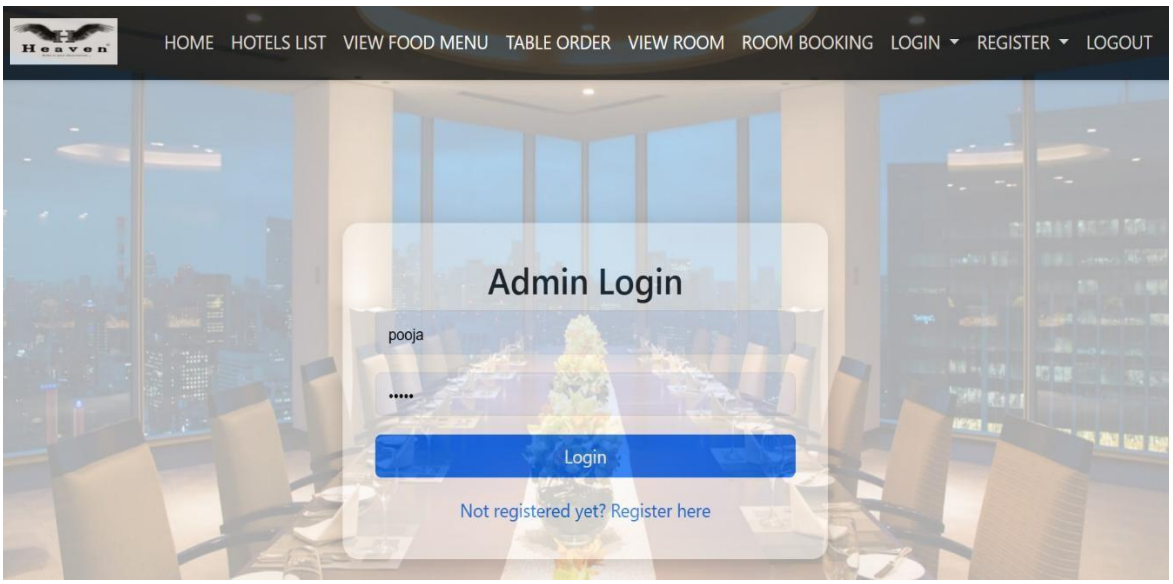


Fig 6.7: Admin login

ADMIN INTERFACE

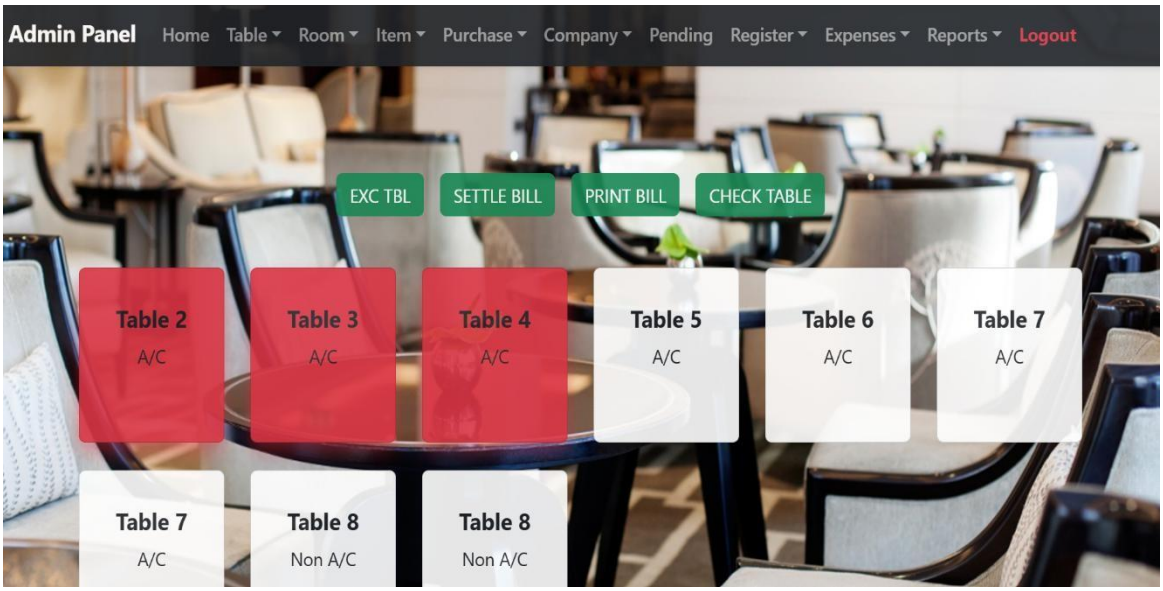


Fig 6.8: Admin interface

USER REGISTER

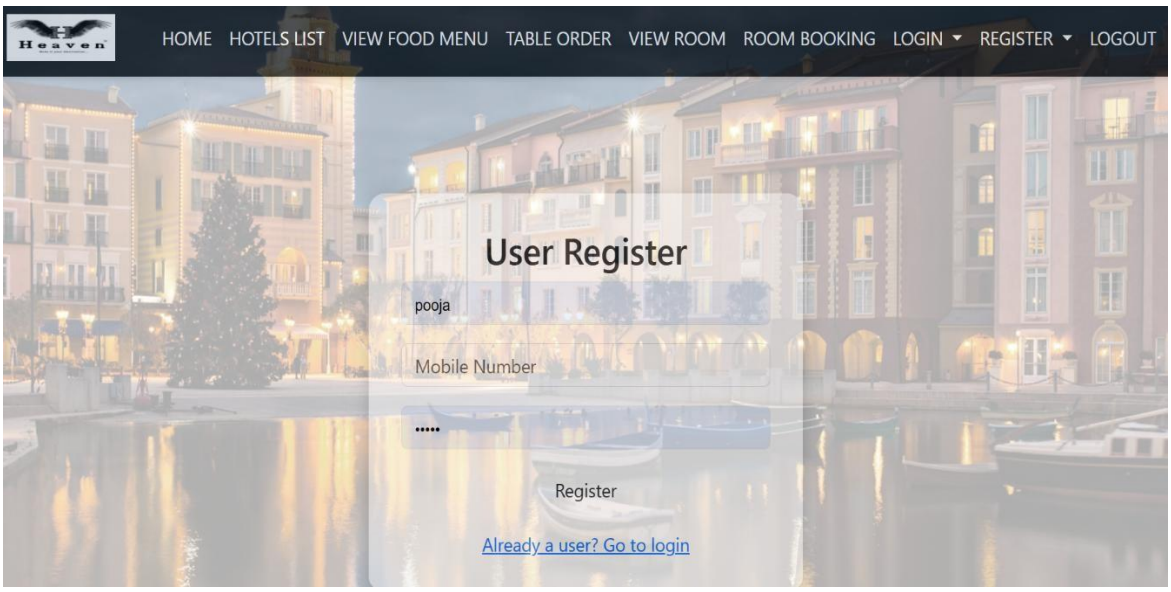


Fig 6.9: User Register

USER LOGIN

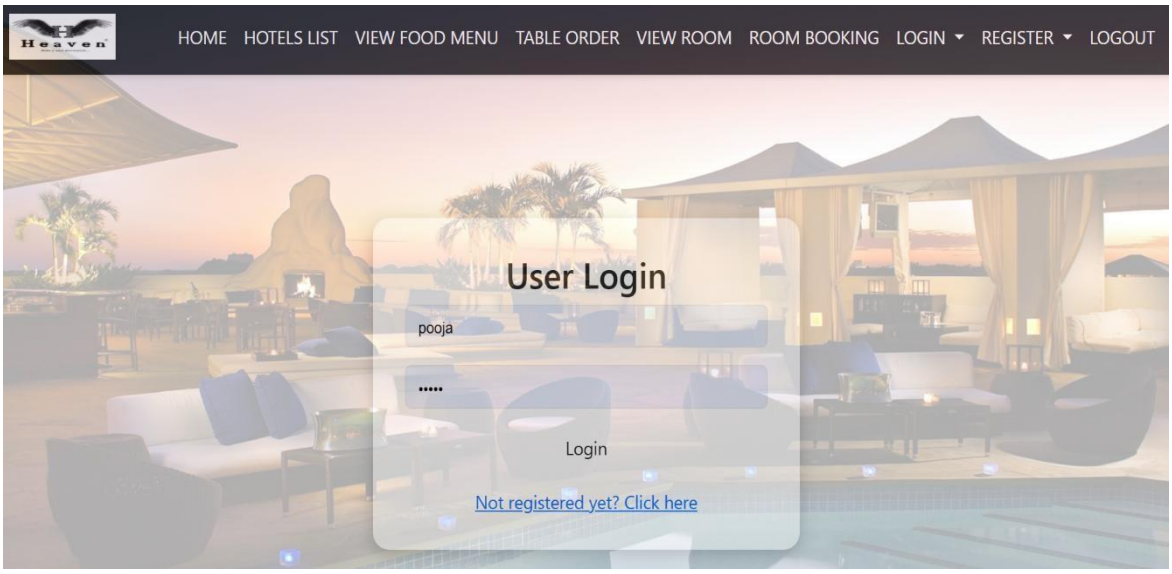


Fig 6.10: User Login

USER TABLE ORDER

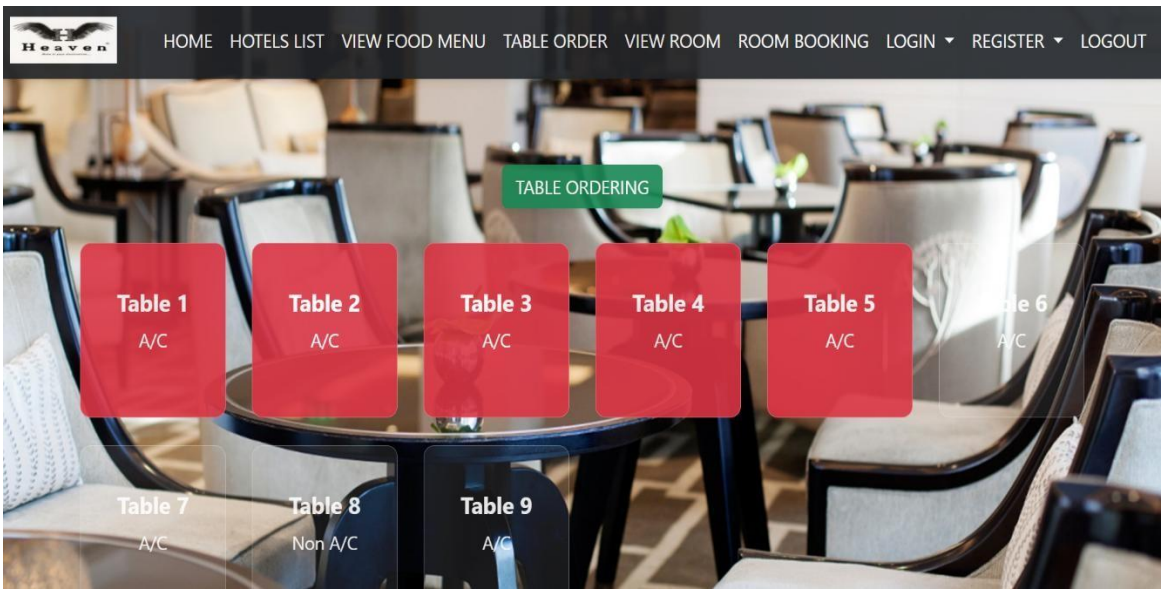


Fig 6.11: Table Order

USER ROOM BOOKING

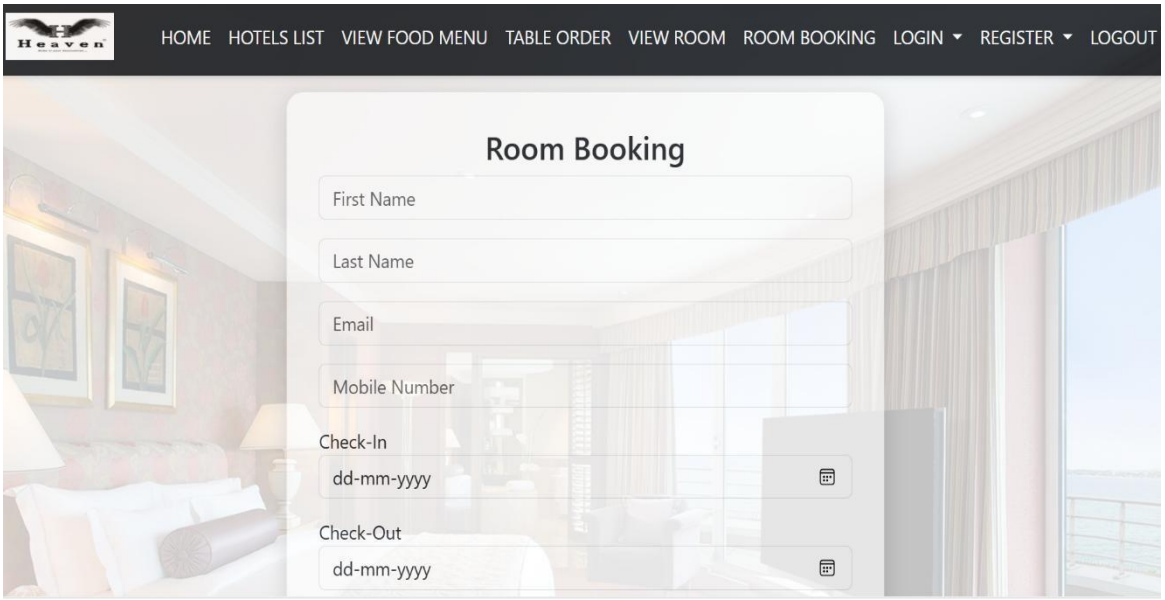


Fig 6.12: Room Book

CONCLUSION

The Hotel Management System is a web-based application developed to eliminate the inefficiencies of manual hotel operations. It addresses issues like booking errors, billing mistakes, and poor inventory tracking through an integrated platform with separate admin and user interfaces. The system includes key modules such as Room Booking, Table Ordering, Staff Management, and Stock Management. Customers can book rooms, select preferences, and place food orders, while admins manage staff, inventory, and billing. Automated calculations, real-time updates, and centralized data help reduce manual workload and errors. Overall, it enhances hotel efficiency, decision-making, and customer satisfaction.

FUTURE ENHANCEMENT

To enhance the Hotel Management System, future upgrades can include integrating online payment gateways for secure transactions and developing mobile apps for greater accessibility. QR code-based table ordering can offer a contactless dining experience, while a feedback system can help improve service quality. Real-time room availability maps and AI-driven recommendations can personalize the user experience. Automated notifications for bookings and offers can boost user engagement. Multi-language support will cater to international guests, and analytics dashboards will aid admins in making informed decisions.

REFERENCES

YouTube

- https://youtu.be/p3eW_jo93RU?si=iVlOmNc2W1vNpeP4
- https://youtu.be/olUBhVOABSQ?si=mpb-Aung-20CmA4u

Websites

- https://www.w3School.com/
- https://www.geeksforgeeks.org/